



Developer Guide

UC-win/Road PythonAPI

UC-win/Road Ver. 19.0.1

株式会社フォーラムエイト
東京都港区港南 2-15-1 品川インターシティ A 棟 21F



Contents

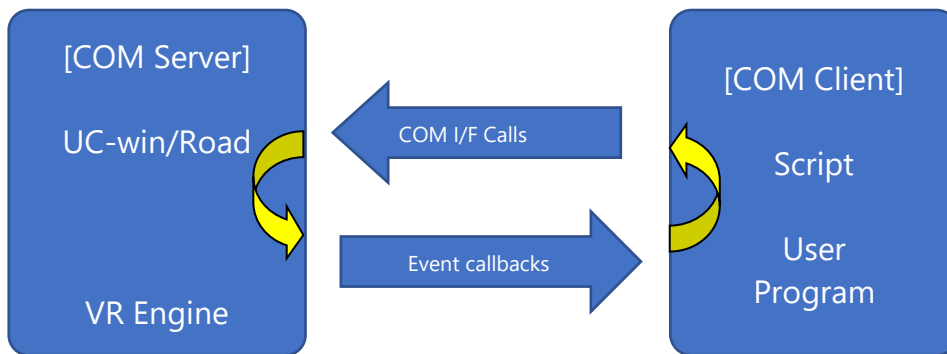
1. UC-win/Road Python API Overview.....	4
1.1. COM INTERFACE.....	4
1.2. API Structure	4
2. Build an environment.....	6
2.1. Install Python	6
2.2. Install Python Packages.....	7
2.3. About the directory where scripts are stored	8
2.4. About Licenses	9
2.5. Registration/deregistration of COM server	9
2.5.1. Registration	9
2.5.2. Deregistration	10
2.5.3. COM Server Version Switching.....	10
3. Python script execution.....	11
3.1. Script execution from UC-win/Road.....	11
3.1.1. Execution from the Python API Ribbon.....	11
3.1.2. Execution from the Python Script Editor.....	12
3.1.3. Running from a Scenario	13
3.1.4. Python Script Manager.....	14
3.2. Script execution from external tools	15
4. Saving Python scripts to RD files	15
5. How to use the API	16
5.1. Dispatch to COM server.....	16
5.1.1. ProgID of UC-win/Road	16
5.1.2. Early Binding.....	16
5.1.3. Late Binding.....	17
5.2. Accessing the Interfaces	17
5.2.1. Methods.....	17
5.2.2. Properties	18
5.2.3. Interface Release.....	19
5.3. callback events	19
5.3.1. callback event interface.....	19
5.3.2. Registering and deregistering callbacks	20
5.3.3. Callback types.....	21
5.3.4. Event loop	22
5.4. Enumerated type reference	23
6. Sample Scripts	24
6.1. Sample_ContextControl	25
6.2. Sample_CoordinateConversion	25
6.3. Sample_GazeTrackReceive	26
6.4. Sample_InputDeviceLog.....	27
6.5. Sample_MainCameraOperation	27
6.6. Sample_OutputVehicleInfo	27

6.7.	Sample_OverrideCarInput	28
6.8.	Sample_OverrideCarMovement	28
6.9.	Sample_RoadInformation	29
6.10.	Sample_SimulationScreen	29
6.11.	Sample_ThreeDModelCommand.....	30
6.12.	Sample_UserVariables.....	30
6.13.	Sample_VehiclePlacement.....	31
6.14.	Sample_VisualOptionControl	31
6.15.	Sample_BrakeAssist	31
6.16.	Sample_ObstructiveDriving	32
6.17.	Sample_DiagnosisDriving	32
6.18.	Sample_VirtualDisplay	33
6.19.	Sample_GPSroads.....	33

1. UC-win/Road Python API Overview

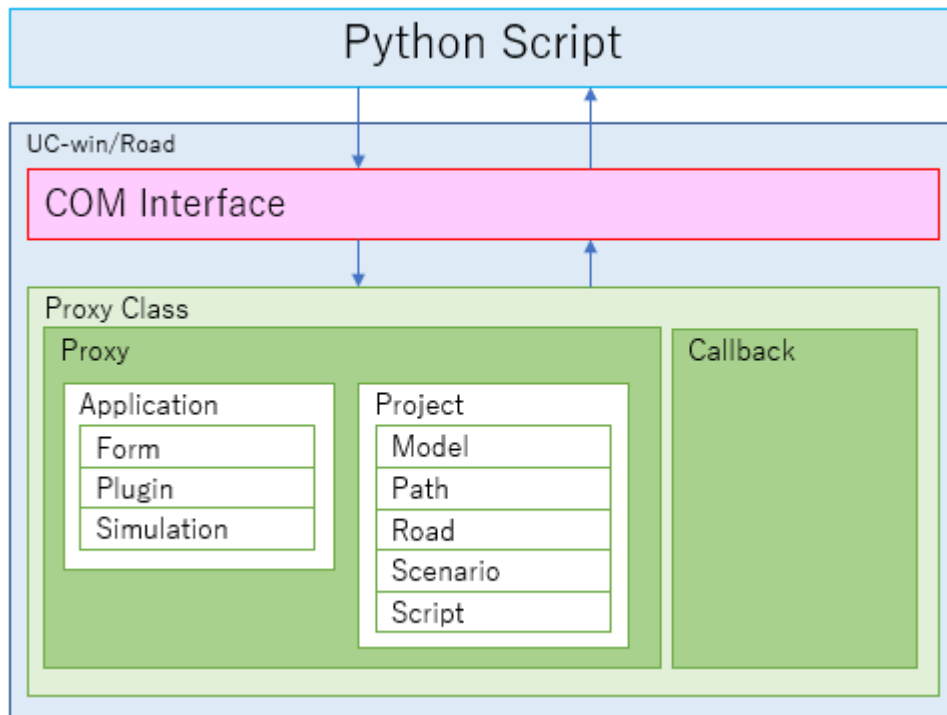
1.1. COM INTERFACE

The UC-win/Road Python API is implemented with a COM interface. COM stands for Component Object Model, Microsoft's binary interface standard for software components, which enables the creation of objects for language-independent inter-process communication on Windows platforms. The Interface provides communication between a COM server and a COM client.



1.2. API Structure

It provides functions equivalent to C++API (Proxy Class) with COM interface.



Please refer to the UC-win/Road Python API reference (UCwinRoad_PythonAPI.chm) for details on the COM interface.

UCwinRoad::IUCwinRoadCom is the COM entry object.

2. Build an environment

This section describes how to build an environment for running Python scripts in UC-win/Road.

2.1. Install Python

Please install Python from the following.

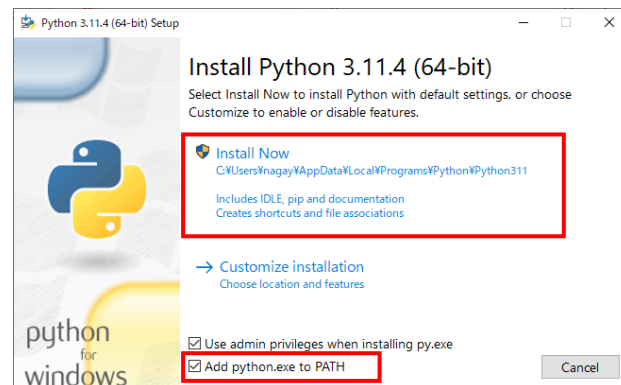
<https://www.python.org/downloads/>

UC-win/Road 19.0.1 supports Python version 3.7 to 3.11.

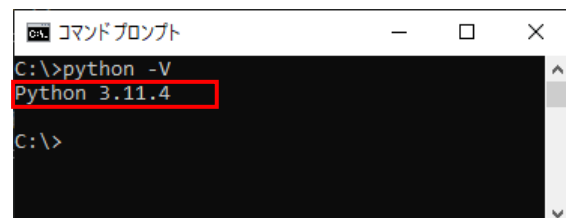
The 32bit/64bit version should match the version of UC-win/Road.

Note:

Check "Add python to PATH" and install.



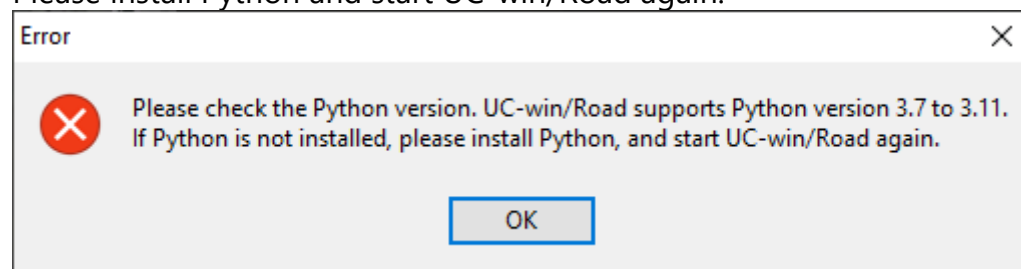
After installation is complete, you can check the Python version with "python -V" at the command prompt.



Note:

If the PC does not have a compatible version of Python installed, the following message will be displayed.

Please install Python and start UC-win/Road again.



2.2. Install Python Packages

Install the package with pip, a system for installing Python packages.

Please see the Python User's Guide for more information on installing packages.

<https://packaging.python.org/en/latest/tutorials/installing-packages/>

To make COM available, install the Python package "pywin32".

Execute the following at the command prompt.

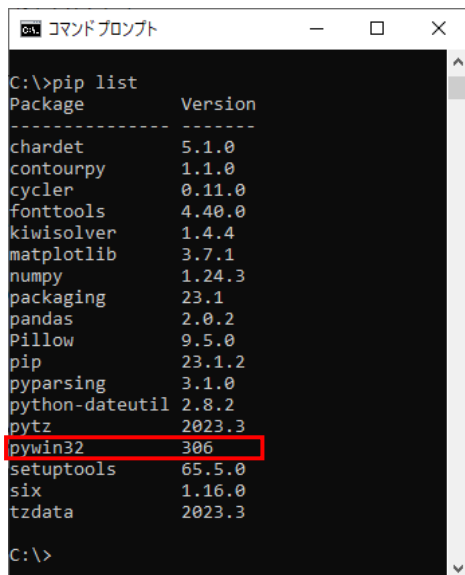
```
pip install pywin32
```

To run the samples included in the UC-win/Road SDK, please also install the Python packages "pandas", "matplotlib" and "PyOpenGL" used in the sample scripts.

Execute the following at the command prompt.

```
pip install pandas  
pip install matplotlib  
pip install PyOpenGL
```

You can run "pip list" at the command prompt to see a list of installed packages.



```
C:\>pip list  
Package          Version  
-----  
chardet          5.1.0  
contourpy        1.1.0  
cyclor           0.11.0  
fonttools        4.40.0  
kiwisolver       1.4.4  
matplotlib       3.7.1  
numpy            1.24.3  
packaging        23.1  
pandas           2.0.2  
Pillow           9.5.0  
pip              23.1.2  
pyparsing        3.1.0  
python-dateutil  2.8.2  
pytz             2023.3  
pywin32          306  
setuptools       65.5.0  
six              1.16.0  
tzdata           2023.3  
C:\>
```

Note:

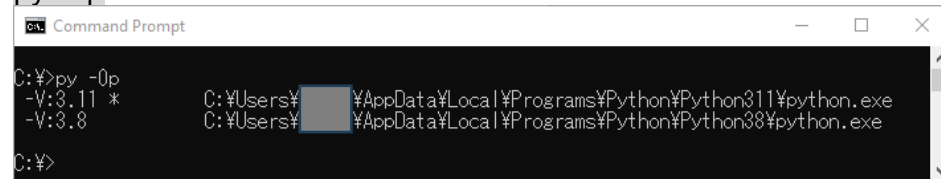
If more than one Python version is installed on the PC, UC-win/Road will use the Python version that is newer.

Python requires packages to be installed for each version, so configure the

environment settings for each version.

Run "py -0p" at the command prompt to see all versions of Python on your PC and their paths.

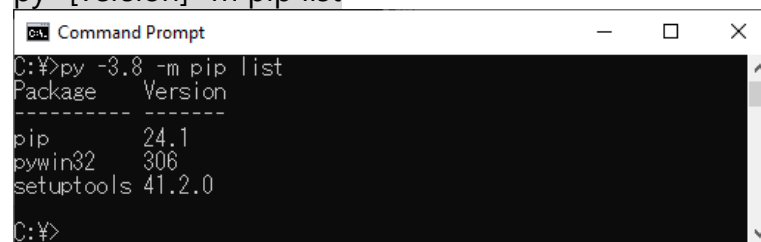
py -0p



```
Command Prompt
C:\>py -0p
-V:3.11 *      C:\Users\%USER%\AppData\Local\Programs\Python\Python311\python.exe
-V:3.8         C:\Users\%USER%\AppData\Local\Programs\Python\Python38\python.exe
C:\>
```

You can check the list of installed packages by specifying the version.

py -[version] -m pip list



```
Command Prompt
C:\>py -3.8 -m pip list
Package      Version
-----
pip          24.1
pywin32      306
setuptools   41.2.0
C:\>
```

You can specify a version and install the package.

py -[version] -m pip install [package name]

2.3. About the directory where scripts are stored

All scripts used in UC-win/Road should be stored in the following directory for loading and executing. All scripts, including the executable script and any user scripts that import into the executable script, should be stored in the following directory.

Storage location: <Data Directory>%Plugins%PythonAPIPlugin

<Data Directory> is the user data directory set in the Application Defaults screen.

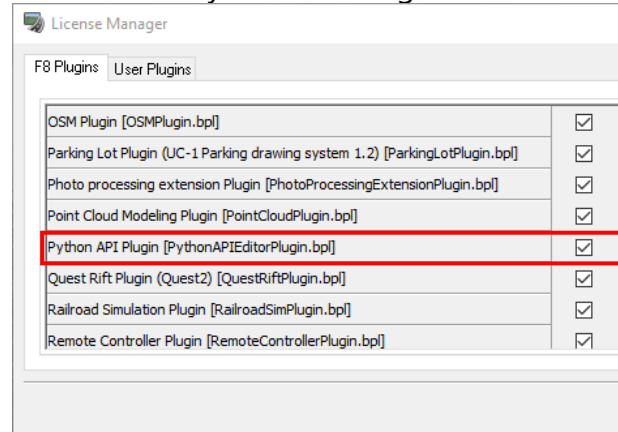
Note:

- For UCwinRoad 17.2.0 or later, the above directory is automatically generated during installation. For earlier versions, users must create the directories manually.
- The reason for limiting the directory where scripts are stored is to enable import of modules and classes in the execution script. In Python, the directory where scripts to be imported in the executable script must be set to the Python path. UC-win/Road automatically sets the above storage directory path to Python at startup.

2.4. About Licenses

To use the Python API, a license for "UC-win/Road SDK (SS)" or "UC-win/Road SDK Suite Bundle (SS)" is required.

Activate the Python API Plugin in the License Manager.



Note:

If the license is invalid, you can still run the Python scripts in the RD file, but you cannot add or edit scripts or use other features.

2.5. Registration/deregistration of COM server

To use this API, UC-win/Road (COM server) must be registered in the system registry on your PC.

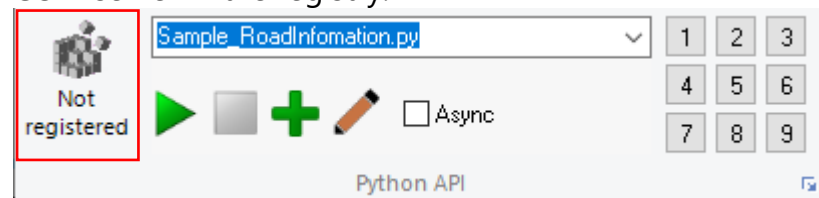
To register or unregister the COM server in the registry, go to the "Python API" tab on the Ribbon menu.

Note:

If the license is invalid, the "Python API" tab will be displayed only when the RD file containing the Python script is opened.

2.5.1. Registration

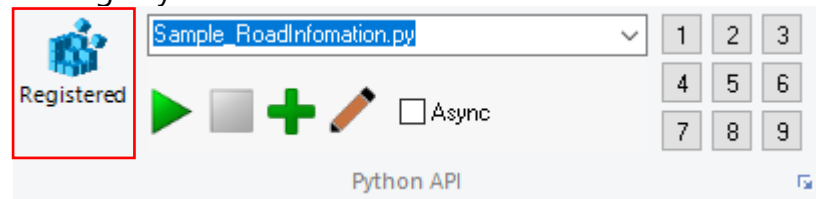
Click on the registry icon "Not registered" in the ribbon menu to register the COM server in the registry.



Once registered, the registry icon will change to "Registered".

2.5.2. Deregistration

If the registry icon is pressed in this state, the COM server will be removed from the registry.



2.5.3. COM Server Version Switching

The UC-win/Road Python API does not allow different versions of COM server registration on a single PC. When switching UC-win/Road versions due to version upgrades, etc., please unregister the COM server from the UC-win/Road version in which the COM server is currently registered and then register the COM server in the new version of UC-win/Road. Also, if you uninstall UC-win/Road, the COM server will be unregistered.

The COM server has a unique ProgID for each version of UC-win/Road. Once the COM server is registered for use with the same version of UC-win/Road, no registration operation is required for subsequent use.

If the version of UC-win/Road is changed, the COM server registration needs to be updated.

Procedure for switching versions:

1. Unregister the COM server from the registered version of UC-win/Road.
2. After that, register the COM server in the version of UC-win/Road to be switched.

Note:

If an error occurs when executing the script during version switching, execute the sample script 'ShowGeneratePath.py' to open the directory where cache information generated by win32com is stored, and delete all folders and files under the directory. The error may be resolved by deleting all folders and files under the directory. The cache information is regenerated when the script is executed, so there is no problem with deleting it.

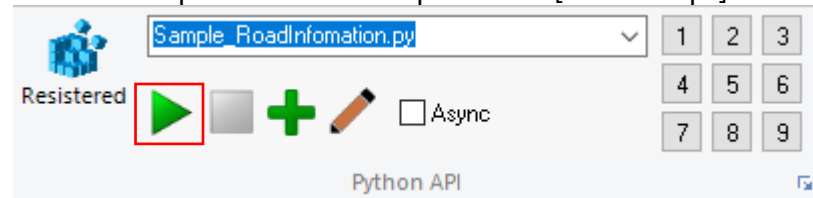
3. Python script execution

3.1. Script execution from UC-win/Road

This section describes the execution of Python scripts from UC-win/Road and related UI operations.

3.1.1. Execution from the Python API Ribbon

Select a script in the list and press the [Start Script] button.

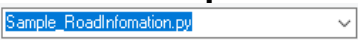


Note:

If the license is valid, the "Python API" tab will always be displayed.

If the license is invalid, the "Python API" tab will be displayed only when the RD file containing the Python script is opened.

3.1.1.1. [Python API] Ribbon Description

- Script selection list: 
You can select a script from the list.
- [Start Script] button: 
Run the currently selected script.
- [Stop Script] button: 
Stops the script execution when the script is running asynchronously.
- [Add New Script] button: 
Loads a new script. Clicking the button opens the load dialog.
Multiple files can be selected.
If the license is invalid, this feature will be disabled and cannot be used.
- [Edit Script] button: 
Opens the [Python Script Editor] screen.
- [Async] checkbox: 
This option allows the script to run asynchronously.
Turn this checkbox ON when executing a script that uses a message loop to wait for event callbacks. When it is turned ON, the script is executed in a separate thread from the main thread.

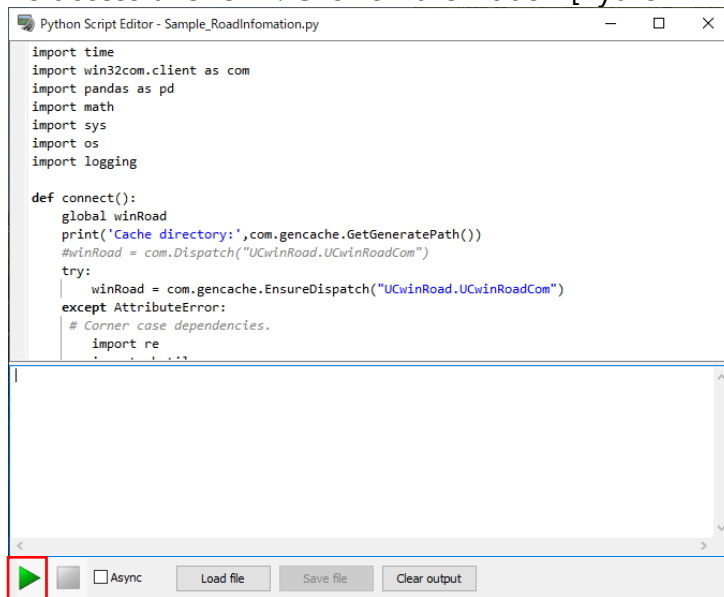
- User Flag button:  ... 
It toggles the "IF8ApplicationServicesProxy.PythonScriptUserFlg" ON/OFF.
This is an optional flag that users can use in their scripts.
- Option Indicator: 
Opens the [Python Script Manager] screen.
If the license is invalid, this feature will be disabled and cannot be used.

3.1.2. Execution from the Python Script Editor

This form for editing, executing, loading and saving Python scripts.

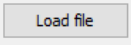
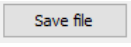
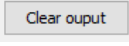
Execute the script with the [Start Script] button.

To access this form: Click on the ribbon [Python API] - [Edit] button.



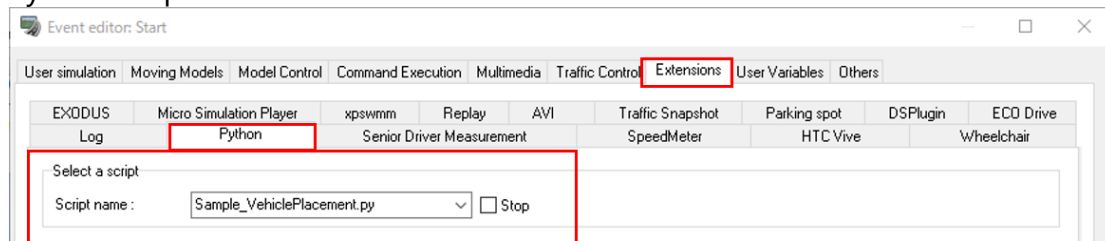
3.1.2.1. UI Description

- Editor section (top of screen):
Displays the content of the currently selected script. It can edit the currently selected script.
If the license is invalid, this feature will be disabled and cannot be used.
- Result output section (bottom of screen):
Displays the result of script execution. If the [Async] check is turned ON, the script result is not displayed. In this case, output the execution result to a file and check it.
- [Start Script] button: 
Executes the script.

- [Stop Script] button: 
Stops the script execution when the script is running asynchronously.
- [Async] checkbox: 
This option allows the script to run asynchronously.
Turn this ON if you want to run the script that uses the message loop to wait for event callbacks.
- [Load file] button: 
Used to load a script.
If the license is invalid, this feature will be disabled and cannot be used.
- [Save file] button: 
Saves the script. Editing the script activates the button.
If the license is invalid, this feature will be disabled and cannot be used.
- [Clear output] button: 
Clears the output section.

3.1.3. Running from a Scenario

Python scripts can be executed from scenario events.



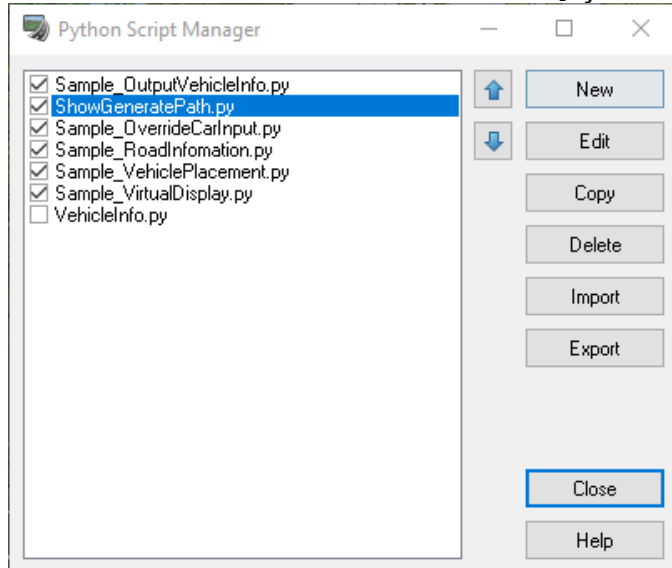
On the "Extensions" tab of the Event editor screen, select the "Python" tab.

- Script Execution:
Select a script in the list and set the Stop check box to OFF.
The selected script will be executed when the event is executed.
The script will be executed with the [Async] check ON (check box in the Ribbon UI, Script Edit screen).
- Stop Script:
Stop check box is turned ON.
The currently running script will be stopped when the event is executed.

3.1.4. Python Script Manager

This form is used to sort, add, edit, copy, delete, import, export Python scripts and switches the selection of scripts to be displayed in the script selection list of the Ribbon menu.

To access this form: Click on the ribbon [Python API] - [Option Indicator] button.



3.1.4.1. UI Description

- New button:
Add a new script.
See the "Python Script Editor" screen for screen operations.
- Edit button:
Edit the selected script.
See the "Python Script Editor" screen for screen operations.
- Copy button:
Copies the selected script.
- Delete button:
Deletes the selected scripts.
Cannot be deleted if a script is set to an event in the scenario.
- Import button:
Loads a script. Clicking the button opens the load dialog.
Multiple files can be selected.
- Export button:
Outputs the selected script to a file as a Python script (*.py).

Clicking this button opens the "Select Folder" dialog.
Select the destination folder and output the file.

- Close button:
Close the Script Manager screen.
- Check box
Selects scripts to be displayed in the script selection list on the Ribbon menu.
Only checked scripts will be displayed in script selection list on the Ribbon menu.
Cannot be hidden if a script is set to an event in the scenario.

3.2. Script execution from external tools

Scripts can be executed from external tools such as Visual Studio Code, etc.
Start UC-win/Road registered as a COM server and execute the scripts with the external tools. Debug execution is also possible with external tools.

Important points:

1. Execution speed is slower than the UC-win/Road.
2. Please use the User Flag button on the Ribbon UI as a trigger to end the loop when the script is in a message loop.

Example:

User Flag ON before execution, User Flag OFF when stopped.

End the loop by judging the user flag OFF in the script.

3. If the license is invalid, script execution from external tools is not possible.

4. Saving Python scripts to RD files

Python scripts loaded into UC-win/Road are saved to an RD file when the project is saved. When the saved RD file is opened, the project starts with the Python scripts loaded. The scripts in the RD file can be executed even if the license is invalid.

5. How to use the API

This chapter describes how to implement Python scripts using the API.

5.1. Dispatch to COM server

Dispatch is performed using ProgID to connect from Python to UC-win/Road (COM server).

There are two types of dispatch methods: early binding and late binding. Early binding is a static binding method at compile time, and late binding is a dynamic binding method at run time.

Note:

Please refer to the sample script "UCwinRoadCOM.py" for a Dispatch to the COM server.

The sample script "ShowGeneratePath.py" opens the directory where the cache information created by the early binding of win32com is stored.

Run it to check if necessary.

5.1.1. ProgID of UC-win/Road

ProgID is unique for each version of UC-win/Road.

```
"UCwinRoad.UCwinRoadCom_AAAA"
```

AAAA: Number set for each version

1st and 2nd digit: Major version

3rd digit: Minor version

4th digit: Patch version

Example: In case of UCwinRoad Ver19.0.1

```
"UCwinRoad.UCwinRoadCom_1901"
```

5.1.2. Early Binding

Early binding creates cached information about object methods, properties, constants and enums before program execution.

Early binding is recommended for UC-win/Road's Python API.

Early binding is forced in the following.

```
import win32com.client as com
winRoad = com.gencache.EnsureDispatch("UCwinRoad.UCwinRoadCom_1901")
```

If there is no cache information, the cache of the bound type library and interface will be output in the following folder.

%LOCALAPPDATA%\Temp\gen_py\%xx%\CLSID

xx: Python version

CLSID: GUID of the COM server type library

You can check the output path with the following:

```
dir_name = com.gencache.GetGeneratePath()
```


5.1.3. Late Binding

If cache information is available, early binding is used; otherwise, late binding is used.

```
winRoad = com.Dispatch("UCwinRoad.UCwinRoadCom_1901")
```

Force late binding.

```
winRoad = com.dynamic.Dispatch("UCwinRoad.UCwinRoadCom_1901")
```

5.2. Accessing the Interfaces

Dispatch to UC-win/Road (COM server) to get the IUCwinRoadCom interface. From this interface, get the IF8ApplicationServicesProxy interface, which is the entry point for the API.

```
app = winRoad.ApplicationServices
```

Similarly, access each interface from the IF8ApplicationServicesProxy interface.

Example:

Get IF8ProjectProxy interface from IF8ApplicationServicesProxy interface

```
proj = app.Project
```

Please refer to the UC-win/Road Python API reference (UCwinRoad_PythonAPI.chm) and the Proxy Class structure detail file **AccessTree.svg** for the order and method of accessing the interface.

5.2.1. Methods

This section describes the interface method calls.

Call the methods of the interface as follows.

Example: IF8RoadProxy interface GetPositionAt ()

```
HRESULT _stdcall GetPositionAt  
([in] double Distance,  
[out, retval] IF8COMdVec3 **Value)
```

```
for i in range(count):  
    road = prj.Road(i)  
    roadLength = road.Length  
    dist = 0.0  
    while dist <= roadLength:  
        roadPos = road.GetPositionAt(dist);
```

Important points

1. When setting the Out parameter of the interface type at the time of method call, set the interface obtained by Dispatch as follows. The ProgID used for Dispatch is 'UCwinroad.XXX', where XXX is the symbolic name of the interface name without the "I" prefix.

```
srcVec2 = com.Dispatch('UCwinRoad.F8COMdVec2')
dstVec2 = com.Dispatch('UCwinRoad.F8COMdVec2')
convRes = com.Dispatch('UCwinRoad.F8COMHcsConvertResultType')
hConverter.Convert(const_hcLocal_XY, const_hcWGS84_LonLat, srcVec2, dstVec2,
convRes);
```

2. Python does not support the concept of pass-by-reference, so simple parameters cannot be passed by reference. If the type of the Out parameter is a simple parameter other than an interface type, the Out parameter will not be updated and will be returned as a return value. Please write the Out parameter as a return value as follows to get the value.

Example: IF8ProjectProxy interface GetActualHeightAt ()

```
HRESULT_stdcall GetActualHeightAt
([in] double X,
 [in] double Y,
 [in] IF8COMHeightSearchConditionsType *conditions,
[out] double *Height,
 [out, retval] VARIANT_BOOL *Value)
```

The following description does not update h in the out parameter.

```
prj.GetActualHeightAt(100, 500, a, h)
```

If you write the following to get the value by return value, "h" will be updated.

The parameters are written in the order of retval and out parameter.

If there are multiple out parameters, describe them in order from left to right.

```
ret, h = prj.GetActualHeightAt(100, 500, a, h)
```

5.2.2. Properties

This section describes the interface properties.

5.2.2.1. Getter

Properties can be obtained as follows.

Example: IF8RoadProxy interface Name and Length properties

```
BSTR Name[get]
double Length[get]

for i in range(count):
    road = prj.Road(i)
    name = road.Name
    roadLength = road.Length
```

5.2.2.2. Setter

Properties can be set as follows.

Example: IF8MainFormProxy interface NavigationState property

IF8NavigationStateType NavigationState[set]

```
naviSt = MainForm.InitialNavigationState
MainForm.NavigationState = naviSt #set
```

Important point:

1. Symbolic names of properties with arguments:

In cache information created by early binding with win32com.client.gencache.EnsureDispatch, the Setter of a property with arguments is converted as a method.

Note that the symbol names are converted to different symbol names from those listed in the UC-win/Road Python API Reference (UCwinRoad_PythonAPI.chm).

Example:

IF8SimulationCoreProxy interface UserVariable property

double UserVariable ([in] int idx)[get, set]

Property with index as an argument

UserVariable->**Set**UserVariable the symbol name is prefixed with **Set**.

When using Setter, use the symbol name that have **Set** as prefix.

```
simCore = app.SimulationCore
#getter
v = simCore.UserVariable(0)
#setter
simCore.SetUserVariable(0, 88)
```

5.2.3. Interface Release

The acquired interface should be released after use by explicitly assigning None.

```
proj = None
app = None
winRoad = None
```

5.3. callback events

This section describes how to receive events from a COM server with a COM client.

5.3.1. callback event interface

Callback event interfaces are provided for interfaces with **Events** appended to the end of the interface, such as IF8ApplicationServicesProxy**Events**. There is also an interface with IF8ObjectProxy**Events**. Please refer to the UC-win/Road Python API reference (UCwinRoad_PythonAPI.chm) for more information on event interfaces.

Example:

UCwinRoad::IF8ApplicationServicesProxyEvents インタフェース

F8ApplicationServicesProxyオブジェクトのイベントインターフェース。[詳解]

公開メンバ関数

HRESULT	OnPluginAbleMenus ([in] VARIANT_BOOL enable) アプリケーションがリボンメニューの有効・無効状態を切り替えるときに呼び出されるコールバック。
HRESULT	OnErrorOrWarning ([in] enum F8COMApplicationErrorTypeEnum errorType, [in] unsigned short errorCode, [in] BSTR errorMessage) エラー、警告、または情報をユーザーに通知する時に呼び出されるコールバック。
HRESULT	OnNewProject (void) プロジェクトを新規作成した時、または開いた時に呼び出されるコールバック。
HRESULT	OnBeforeSaveProject ([in] BSTR Name) プロジェクトの保存前に呼び出されるコールバック。
HRESULT	OnAfterSaveProject (void) プロジェクトの保存後に呼び出されるコールバック。
HRESULT	OnCloseProjectQuery ([out, retval] VARIANT_BOOL *allow) プロジェクトを閉じる前に確認を行うコールバック。
HRESULT	OnBeforeDestroyProject (void) プロジェクトを解放する前に行うコールバック。
HRESULT	OnAfterLoadProject (void) プロジェクトを読み込んだ後に呼び出されるコールバック。
HRESULT	OnIsExistEventHandler ([in] BSTR funcname, [out, retval] VARIANT_BOOL *Value) イベントハンドラクラスにコールバック関数が存在するか確認を行うコールバック。

5.3.2. Registering and deregistering callbacks

This section describes how to register a callback to a COM server in a Python script.

5.3.2.1. Definition of Event Handler Class

Example: Definition of the event handler class ApplicationServicesHandler

```
class ApplicationServicesHandler:
    def SetCOMEventClass(self, events):
        self.events = events
    def OnIsExistEventHandler(self, funcname):
        try:
            func = getattr(self.events, funcname)
        except AttributeError:
            return False
        return True
    def OnPluginAbleMenus(self, enable):
    def OnErrorOrWarning(self, errorType, errorCode, errorMessage):
    def OnNewProject(self):
    def OnBeforeSaveProject(self, name):
    def OnAfterSaveProject(self):
    def OnCloseProjectQuery(self, allow=pythoncom.Missing):
    def OnBeforeDestroyProject(self):
    def OnAfterLoadProject(self):
```

Description

def OnPluginAbleMenus ... def OnAfterLoadProject(self):
Implement the necessary event callback functions.

def SetCOMEventClass(self, events):

def OnIsExistEventHandler(self, funcname):

These functions are used to check for the existence of a callback function from the COM server when a COM client registers a handler function for a callback event to the COM server. The COM server executes the handler function registration process only when the callback function exists.

Be sure to implement the above two functions by copying the sample.

5.3.2.2. Handler Registration to Interface

Follow the steps below to register.

1. Set the event handler class for callback to the target interface with win32com.client.WithEvents().
2. Set its own Event information to win32com.client.COMEventClass with SetCOMEventClass.
3. Register callback event handler function with RegisterEventHandlers() of the target interface.

Example: Event handler for IF8ApplicationServicesProxy

Setup of ApplicationServicesHandler

```
import win32com.client as com
app = winRoad.ApplicationServices
appEvent = com.WithEvents(app, ApplicationServicesHandler)
appEvent.SetCOMEventClass(appEvent)
app.RegisterEventHandlers()
```

5.3.2.3. Unregistering a handler from the interface

Follow the steps below to unregister the handler.

1. Unregister the callback event handler class with win32com.client.COMEventClass.close ().
2. Unregister the callback event handler functions with UnRegisterEventHandlers() of the target interface.

```
appEvent.close()
app.UnRegisterEventHandlers()
```

5.3.3. Callback types

There are two types of callback registration functions in UC-win/Road (normal and unique). The normal type allows multiple callbacks to be registered for an event. The unique type allows only one callback to be registered for an event.

If the callback type is unique, it is necessary to check if a callback function has already been registered, and if so, call unregister before registering a new function. For normal callbacks, there is no need to check.

Unique types have the following functions in the target interface.

Registration confirmation: begins with "**IsSetCallback**".

Unregistration: begins with "**UnsetCallback**".

Example: IF8TransientInstanceProxy interface:

IsSetCallbackOnCalculateMovement()

To register or unregister a callback, call the same function, both normal and unique.

Register: **RegisterEventHandlers()**

Unregister: **UnRegisterEventHandlers()**

If a callback function has already been registered with a unique type, calling RegisterEventHandlers() will update it, and calling UnRegisterEventHandlers() will unregister it.

5.3.4. Event loop

Implement a message loop to receive callbacks from the COM server.

Example: Message loop (loop control by user flags)

```
loopFlg = True
app.IsPythonScriptRun = loopFlg
while loopFlg:
    time.sleep(0.0005)
    loopFlg = app.PythonScriptUserFlg(0)
    if loopFlg == False:
        logger.info("loopFlg={}".format(loopFlg))
        logger.info("Script close")

app.IsPythonScriptRun = loopFlg
```

5.3.4.1. Notes on script stops

To properly terminate the event loop, you need to terminate the script from Python. Please note that if you stop the loop from the Python script and do not unregister the event handler, the callback event will continue.

The following two methods are provided:

1. Use the Stop button on UC-win/Road's Ribbon UI and Script Editor.
Please refer to ON/OFF of the property IsPythonScriptRun (Python script execution state) of IF8ApplicationServicesProxy to terminate the event loop.

2. Use the User flag button on the UC-win/Road ribbon UI.
Please refer to ON/OFF of the property PythonScriptUserFlg (user flag) of IF8ApplicationServicesProxy and terminate the event loop.

5.4. Enumerated type reference

You can use win32com.client.constants to refer the COM server enums. This requires an early binding to the COM server.

Call win32com.client.constants to create a container for COM constants.

```
const = win32com.client.constants
```

Example: Use of F8COMTransientTypeEnum._TransientCar

```
if const._TransientCar == transientType :
```

All enumerated types used in the Python API are defined by consecutive values starting from 0.

6. Sample Scripts

The UC-win/Road SDK includes the following sample scripts in Python¥Samples to help those who use the Python API.

To run the sample scripts, refer to the chapter on Build an environment.

No.	File name	Description
1	Sample_ContextControl.py	Context switching
2	Sample_CoordinateConversion.py	Coordinate transformation
3	Sample_GazeTrackReceive.py	Receives and calculates data with Gaze Tracking linkage function.
4	Sample_InputDeviceLog.py	Receives mouse, keyboard, and joystick input.
5	Sample_MainCameraOperation.py	Specify the viewpoint position and viewing direction of the main camera.
6	Sample_OutputVehicleInfo.py	Obtains information on traffic vehicles around the camera position and outputs the file
7	Sample_OverrideCarInput.py	Override steering input, accelerator, brake, and clutch pedal input of the driving vehicle
8	Sample_OverrideCarMovement.py	Override the driving vehicle behavior calculation and give the vehicle attitude information
9	Sample_RoadInformation.py	Obtain road information and output to file
10	Sample_SimulationScreen.py	Change the simulation screen settings of the main screen to change the angle of view and display position of the screen.
11	Sample_ThreeDModelCommand.py	Send commands for movable settings to the placed 3D model
12	Sample_UserVariables.py	Get and change user variables
13	Sample_VehiclePlacement.py	Generate a new vehicle object on the simulation
14	Sample_VisualOptionControl.py	Get and control visual options
15	Sample_BrakeAssist.py	Simple assist braking operation by manipulating the accelerator pedal and brake pedal inputs of the driving vehicle
16	Sample_ObstructiveDriving.py	Simple obstructive driving behavior by manipulating the steering input and the accelerator pedal and brake pedal inputs of the vehicle in front of the driving vehicle
17	Sample_DiagnosisDriving.py	Simple driving diagnosis of the driving vehicle
18	Sample_VirtualDisplay.py	Directly draw on the virtual display using OpenGL according to the vehicle's speed.
-	UCwinRoadCOM.py	Sample of Class for COM server access
-	UCwinRoadUtils.py	Utils function sample

-	CallbackHandlers.py	Base class for callback event handler class, sample event class template for each interface class
-	LoggerProxy.py	Sample for log output
-	VehicleInfo.py	Sample dataclass used in Sample_OutputVehicleInfo.py
-	ShowCurrentPath.py	Sample of opening the current directory
-	ShowGeneratePath.py	Sample of opening the directory for storing cache information created by win32com's Early Binding

6.1. Sample_ContextControl

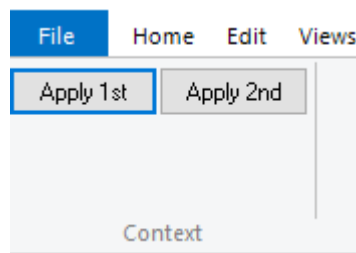
Sample_ContextControl is a script that allows you to experiment with context switching behavior.

When executed with the [Async] checkbox ON, the group "Context" is added to the tab "Python API Samples" in the ribbon menu.

Clicking the button "Apply 1st" will apply the first context registered in the project, and the setting values determined by the context will be applied to Visual options, etc.

Clicking the button "Apply 2nd" applies the second context registered in the project, and the values determined by the context are applied to the Visual options, etc.

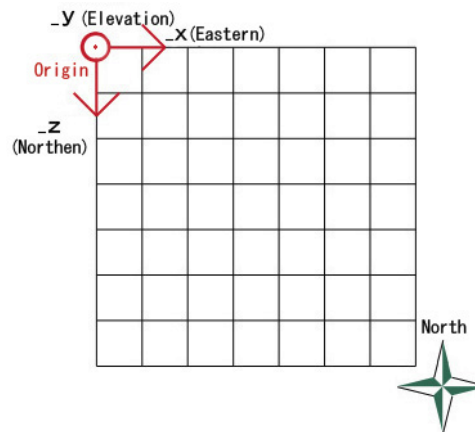
If the context does not exist in the project, it will not change, so please load sample data such as Mt. Parkway.



Contexts are settings used to switch between visual options and simulation environment settings. See UC-win/Road help and operational guidance for how to create and use them.

6.2. Sample_CoordinateConversion

Sample_CoordinateConversion is a script that allows you to try out the coordinate conversion feature. The coordinate system handled by the API is an OpenGL coordinate system as below.



OpenGL coordinate system

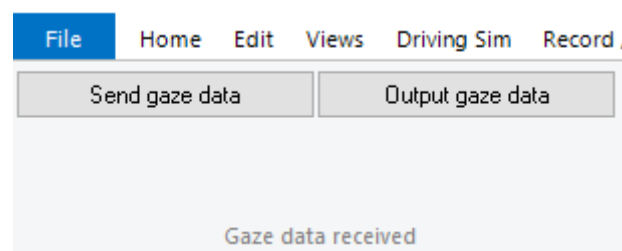
The coordinate converter provides the functionality to convert between the OpenGL coordinate system, the local coordinate system displayed on the program, the global coordinate system, and latitude and longitude.

When executed, it outputs a single local coordinate conversion result and multiple local coordinate conversion results to the result output column of the Script Editor screen and to a log file. (Output is directed to the current directory.)

To convert OpenGL coordinate values, local coordinate values, and global coordinate values to longitude and latitude, a coordinate system must be set in the project, and the coordinate system is assigned when the project is created.

6.3. Sample_GazeTrackReceive

Sample_GazeTrackReceive is a script that allows you to check the operation of receiving data received and calculated by the Gaze tracking linkage function. When executed with the [Async] checkbox ON, the group "Gaze data received" is added to the tab "Python API Samples" in the ribbon menu.



When the gaze tracking plug-in is enabled and gaze measurement data is being received via UDP communication, etc., clicking the button "Send gaze data" and then clicking "Output gaze data" will output the received data and calculated values to a CSV file.

Please refer to UC-win/Road Help for more information on the gaze tracking plug-in.

6.4. Sample_InputDeviceLog

Sample_InputDeviceLog is a script that receives mouse, keyboard, and joystick inputs.

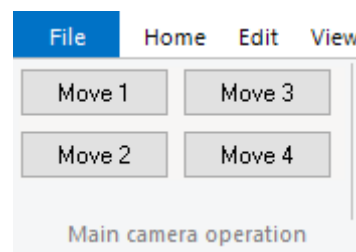
When executed with the [Async] checkbox ON, it receives mouse, keyboard, and joystick inputs and outputs a log file.

(Output will be directed to the current directory.)

6.5. Sample_MainCameraOperation

Sample_MainCameraOperation is a sample script that specifies the viewpoint position and viewing direction of the main camera.

When executed with the [Async] checkbox ON, the group "Main camera operation" is added to the tab "Python API Samples" in the ribbon menu.



Click the "Move 1" button to set the direction of the camera using the position of eyepoint, position of gaze point and tilt angle of the camera.

Click the "Move 2" button to set the direction of the camera using the position of eyepoint, position of gaze point and up vector of the camera.

Click the "Move 3" button to set the direction of the camera using the camera posture matrix.

Click the "Move 4" button to set the direction of the camera using the model view matrix (Inverse matrix of camera posture).

In addition, when using a matrix, it is applied in a form that can be applied to the camera parameters of the UC-win/Road. For example, if the X, Y, and Z axes do not intersect perpendicularly, if you specify a posture matrix or a matrix that changes the scale, it will not be reflected as it is.

6.6. Sample_OutputVehicleInfo

Sample_OutputVehicleInfo is a script that get information of traffic vehicles around the camera position and outputs it to a file.

When executed, it gets the traffic vehicles within 100m around the camera position and outputs various vehicle information to the result output section (bottom of screen) of the Python Script Editor screen and to a CSV file (output

to the current directory). (Output will be directed to the current directory.)

	A	B	C	D	E	F	G	H	I	J	K
1	Time stamp	ID	Description	Position X	Position Y	Position Z	Yaw angle	Pitch angle	Roll angle	Direction X	Direction Y
2	47:17.4	32752782	Car coupe	8545.964	273.8677	12225.96	1.301261	-0.00912	0.001381	0.963855	0.009114
3	47:17.4	32752783	Car coupe	8521.056	273.6497	12221.16	1.458594	-0.0081	-0.00074	0.993679	0.008096
4	47:17.4	32752784	Car coupe	8504.335	273.4829	12219.11	1.454315	-0.01049	-0.01715	0.993169	0.010488
5	47:17.4	32752785	Car coupe	8440.284	273.1714	12222.6	-1.24236	0.005167	0.000081	-0.94653	-0.00517
6	47:17.4	32752786	Car coupe	8471.725	273.3362	12219.17	4.595261	0.005066	-1E-06	-0.99314	-0.00507
7											
8											
9											

6.7. Sample_OverrideCarInput

Sample_OverrideCarInput is a script that performs operations to override the driving vehicle's steering input, accelerator, brake, and clutch pedal inputs.

When the [Async] check box is turned ON and the script is executed after starting driving, the input amount of the driving vehicle is overwritten.

Since the script overwrites the steering input, accelerator, brake, and clutch pedal inputs, these inputs cannot be operated from the keyboard or other devices.

This sample only overwrites the inputs.

If you want to override the results of the vehicle calculation and give the vehicle position, etc. directly, check the Sample_OverrideCarMovement sample.

6.8. Sample_OverrideCarMovement

Sample_OverrideCarMovement is a script that overrides the driving vehicle behavior calculation and gives the vehicle attitude information.

Please follow the steps below to create a project before trying this plugin.

1. Create a new default project.
2. Open the **Plan View** form, select Make Road, click 2 points on the terrain, right-click and select Finish Road.
3. Edit the starting point of the road and set the local coordinates to X=4900 and Y=5000.
4. Edit the second and last point of the road and set the local coordinates to X=5100 and Y=5000.
5. Open the road in Vertical Curve Editor, Open the Vertical Turning Point Editor at start point and make sure the height is set to 560m.
6. Open the Vertical Turning Point Editor at the end point, set the slope to 0%.
7. Close the Plan view screen with OK button.

After starting the driving, turn ON the [Async] checkbox and run the script, which, when clicked, will override the behavior calculation of the driving vehicle. Override the vehicle's coordinates, speed, angle of rotation, direction, body direction, engine RPM, etc. to control the vehicle so that it behaves in a zig-zag manner.

This sample overwrites the behavior calculation.

If you want to override the vehicle input values and perform the behavior

calculation on the UC-win/Road side, please check Sample_OverrideCarInput sample.

6.9. Sample_RoadInformation

Sample_RoadInformation is a script that acquires road information and outputs it to a file.

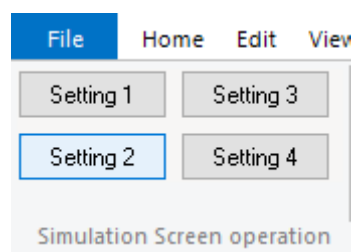
When executed, it retrieves the roads in the project, obtains the coordinates, tangent direction, curvature, height, and slope of the road at 150m pitch, and outputs the data to a CSV file.

	A	B	C	D	E	F	G	H	I	J
1		Name	Length	Distance	Pos X	Pos Z	Direction	Curvature	Height	Slope
2	0	Nihondaira Park Way	1935.578	0	8580.92	12229.95	276.6694	INF	274.23	-1.088
3	1	Nihondaira Park Way	1935.578	150	8432.941	12223.8	241.2964	-60	273.135	-0.499
4	2	Nihondaira Park Way	1935.578	300	8434.858	12354.7	152.5058	INF	271.2889	-1.588
5	3	Nihondaira Park Way	1935.578	450	8406.257	12455.56	293.0124	40.6	265.6421	-8.27529
6	4	Nihondaira Park Way	1935.578	600	8329.973	12543.27	157.8223	-65	253.3872	-7.56674
7	5	Nihondaira Park Way	1935.578	750	8352.215	12672.32	232.1236	60	241.1352	-6.00947
8	6	Nihondaira Park Way	1935.578	900	8219.878	12742.83	242.3144	INF	241.1165	1.063317
9	7	Nihondaira Park Way	1935.578	1050	8087.657	12735.96	307.4846	INF	234.677	-7.68359
10	8	Nihondaira Park Way	1935.578	1200	7948.251	12694.51	275.1296	INF	223.4203	-6.72847
11	9	Nihondaira Park Way	1935.578	1350	7872.668	12597.17	329.6563	-31.2	213.4216	-6.665
12	10	Nihondaira Park Way	1935.578	1500	7805.383	12680.82	181.3717	-50	203.4241	-6.665
13	11	Nihondaira Park Way	1935.578	1650	7860.613	12819.64	156.5292	INF	196.8588	-0.29626
14	12	Nihondaira Park Way	1935.578	1800	7957.275	12927.61	139.7479	28.5	195.8733	-0.86589
15	13	Nihondaira Park Way	1935.578	1935.578	7866.59	12958.69	283.586	INF	194.4361	-1.132

6.10. Sample_SimulationScreen

Sample_SimulationScreen is a script that changes the simulation screen settings of the main screen and changes the angle of view and display position.

When executed with the [Async] checkbox ON, the group "Simulation Screen operation" is added to the tab "Python API Samples" in the ribbon menu.



Click the "Setting 1" button to set the simulation screen according to the window size.

Click the "Setting 2" button to set the simulation screen according to a physical size.

Click the "Setting 3" button to set the simulation screen by Direct FOV.

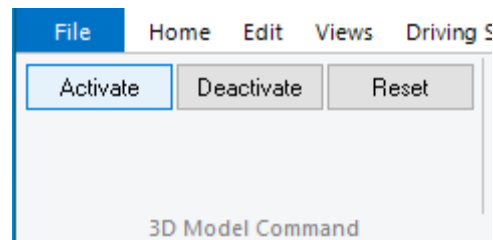
Click the "Setting 4" button to get the simulation screen settings and export them to a CSV file.

6.11.Sample_ThreeDModelCommand

Sample_ThreeDModelCommand is a script that sends commands for movable settings to a placed 3D model.

To try this plugin, load the sample project Highway.rd, select "4D Simulation" from the combo box under Home > Camera Position in the ribbon menu, and move the camera to the crane position.

Turn ON the [Async] checkbox, run it, and the group "3D Model Command" will be added to the tab "Python API Samples" in the Ribbon menu.

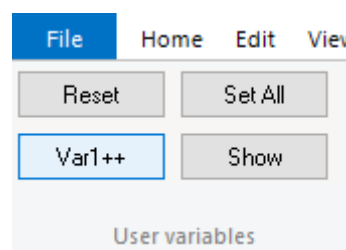


Click the "Activate" button to send the command "right" to all the 3D model instances of the project. As a result, the crane near the camera position "4D simulation" rotates to the right. Clicking the "Deactivate" button during the rotation stops the command "right" for all the model instances of the project. This stops the operation if the crane is spinning. Click the "Reset" button to send the command "*RESET*" to all the model instances of the project. This will restore the models to their original state.

6.12.Sample_UserVariables

Sample_UserVariables is a script to retrieve and modify user variables.

When run with the [Async] checkbox ON, the group "User variables" is added to the tab "Python API Samples" in the ribbon menu.



Clicking the button "Reset" clears the user variable and all user variables are set to 0.

Clicking the button "Var1++" adds 1 to the user variable at index 1.

Clicking the button "Set All" sets all user variables at index 0 to 511 to the same value as the index number.

Clicking the button "Show" outputs the values of the user variables for indexes 0 to 511 to the log file. (The output will be in the current directory)

User variables can be used in the event exit conditions of a scenario.

Even complex exit conditions that cannot be expressed in the event exit conditions of a scenario can sometimes be realized by script-based user variable control and event exit conditions by user variables.

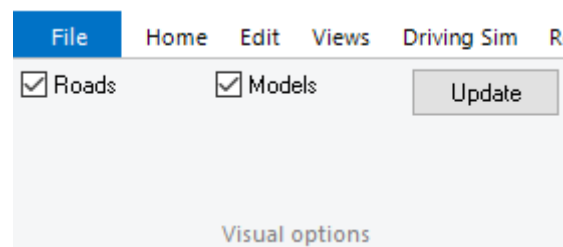
6.13. Sample_VehiclePlacement

Sample_VehiclePlacement is a script that creates a new vehicle object on the simulation. Please execute this script on Mt Parkway.rd.

When executed, the new traffic flow vehicle will be placed in the left-most lane of the left carriageway and the right carriageway near the start of the road "Nihondaira Park Way".

6.14. Sample_VisualOptionControl

Sample_VisualOptionControl is a script to get and control the Visual options. When executed with the [Async] checkbox ON, the group "Visual options" is added to the tab "Python API Samples" in the ribbon menu.



The checked state of the checkboxes is not updated after the project is loaded or when it is changed on the Visual options screen. It is updated by pressing the button "Update", and the current Visual options status is reflected in the checkboxes. Toggling the checkboxes "Roads" and "Models" toggles the display status of roads and models.

6.15. Sample_BrakeAssist

Sample_BrakeAssist is a script that performs simple assist braking operation by manipulating the accelerator pedal and brake pedal inputs of the driving vehicle. It applies the brakes to the driving vehicle when the distance between the driving vehicle and the vehicle in front is less than the predefined required distance for the speed of the driving vehicle.

After starting the driving trip, turn on the asynchronous check box and execute the script. When the assisted braking is in operation, the user flag button "1" on the ribbon UI will be in the pressed state (black).

(In this script, the distance between vehicles is not determined at intersections.)

6.16.Sample_ObstructiveDriving

Sample_ObstructiveDriving is a script that performs simple obstructive driving behavior by manipulating the steering input and the accelerator pedal and brake pedal inputs of the vehicle in front of the driving vehicle.

When the distance between the driving vehicle and the vehicle in front of the driving vehicle is less than the predefined required distance for the speed of the driving vehicle for a certain period of time (2 seconds), the vehicle in front of the driving vehicle performs obstructive driving by meandering and frequent braking.

After starting the driving run, turn on the asynchronous check box and run the script. When the obstructive driving is in operation, the user flag button "2" on the ribbon UI will be pressed (black).

(In this script, the distance between vehicles is not determined at intersections.)

6.17.Sample_DiagnosisDriving

Sample_DiagnosisDriving is a script that performs a simple driving diagnosis of the driving vehicle. It diagnoses whether the distance between the driving vehicle and the vehicle in front is less than the predefined required distance for the speed of the driving vehicle.

It outputs the "diagnosis result (var1)," "elapsed time (var2)," "speed (var3)," "required distance (var4)," "current distance (var5)," and "ID of vehicle in front (var6)" to the user variables. If no diagnosis is performed, each user variable is output as "-1". User variables can be logged by the log output plug-in.

After starting driving, turn on the asynchronous check box and run the script. When the required distance is less than the required distance, the user flag button "3" on the ribbon UI will be pressed (black).

(In this script, the distance is not determined at intersections.)

User Variable Name	Variable Meaning	Value Description
Var1	Diagnosis result	0:OK、 1:NG、 -1: Not diagnosed
Var2	Elapsed time (ms)	-1: Not diagnosed
Var3	Speed(km/h)	-1: Not diagnosed
Var4	Required distance (m)	-1: Not diagnosed
Var5	Current distance (m)	-1: Not diagnosed
Var6	ID of vehicle in front	-1: Not diagnosed

U	V	W	X	Y	Z
var 1	var 2	var 3	var 4	var 5	var 6
-1	-1	-1	-1	-1	-1
1	0	32.096	14	6.109683	32753255
1	0	32.096	14	6.109683	32753255
1	400.1058	26.54608	10	6.892321	32753255
1	749.9972	28.65553	10	8.114516	32753255
1	1152.677	30.1774	14	9.241935	32753255
1	1483.507	31.54796	14	10.18298	32753255
1	1816.763	33.12138	14	11.19318	32753255
1	1816.763	33.12138	14	11.19318	32753255
1	2183.42	34.35972	14	12.31394	32753255
1	2566.837	35.94196	14	13.18159	32753255
-1	-1	-1	-1	-1	-1
0	0	38.42177	14	14.92424	32753255
0	0	39.88333	14	15.58882	32753255
0	0	40.99616	14	16.15722	32753255
0	0	41.98672	14	16.54207	32753255
0	0	43.41881	14	16.83309	32753255

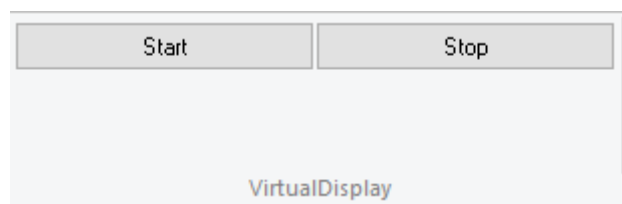
6.18. Sample_VirtualDisplay

Sample_VirtualDisplay is a script that draws shapes from the script to the virtual display using OpenGL. Set up any virtual display in UC-win/Road, start the driving, switch on the asynchronous check box and run the script.

Clicking the 'Start' button in the group 'Virtual Display' in the tab 'Python API Samples' in the ribbon menu draws line segments according to vehicle speed on the virtual display. Clicking the 'Stop' button returns to the original virtual display drawing.

Note:

OpenGL rendering is not possible when running from an external tool. Please run the script from UC-win/Road.



6.19. Sample_GPSroads

Sample_GPSroads is a script that draws shapes from the script to the virtual display using OpenGL. Set up any virtual display in UC-win/Road, start the driving, switch on the asynchronous check box and run the script.

Clicking the 'Start' button in the group 'Sample_GPSroads' in the tab 'Python API Samples' in the ribbon menu draws car and roads on the virtual display. Clicking the 'Stop' button returns to the original virtual display drawing.

You can control the map by keyboard. This is the Keyboard map:

「Home」 Key : Up

「End」 Key : Down

「Delete」 Key : Left

「Page Down」 Key : Right

「Insert」 Key : Zoom In

「Page Up」 Key : Zoom Out

Note:

The plugin supports several virtual displays, and the keyboard will control all displays at once.

OpenGL rendering is not possible when running from an external tool. Please run the script from UC-win/Road.

